

ASSIGNMENT 8 — Data-Driven Automation

1. Experiment Title

Data-Driven Login Automation Using Selenium and PyTest

2. Problem Statement

Develop a data-driven login automation script that reads multiple test cases from an external data source such as a CSV or JSON file. Execute different combinations of valid and invalid usernames and passwords and verify the corresponding login results.

3. Objective

- To implement data-driven testing.
- To read test data from an external file.
- To execute multiple login combinations.
- To verify successful and unsuccessful login attempts.
- To reduce duplication in test scripts.

4. Tools, Software and Concepts Used

Tools & Software:

- Python
- Selenium WebDriver
- PyTest
- Pandas
- CSV
- Google Chrome
- VS Code / PyCharm

Concepts:

- Data-Driven Testing
- External Test Data
- CSV Files

- Pandas
- Selenium WebDriver
- Loops
- Assertions
- Login Validation

5. Source Code / Implementation Steps

Project Structure

Assignment_8_DDT

|

|— test_data.csv

|— test_assignment8.py

test_data.csv

username,password,result

standard_user,secret_sauce,success

wrong_user,wrong_pass,error

standard_user,wrong_pass,error

wrong_user,secret_sauce,error

test_assignment8.py

```
import pandas as pd
```

```
from selenium import webdriver
```

```
from selenium.webdriver.common.by import By
```

```
def test_login_data_driven():
```

```
    data = pd.read_csv("test_data.csv")
```

```
    for _, row in data.iterrows():
```

```
        driver = webdriver.Chrome()
```

```
        driver.maximize_window()
```

```

driver.get("https://www.saucedemo.com/")

driver.find_element(By.ID, "user-name").send_keys(row["username"])

driver.find_element(By.NAME, "password").send_keys(row["password"])

driver.find_element(By.XPATH, "//*[@type='submit']").click()

if row["result"] == "success":

    assert "inventory.html" in driver.current_url

elif row["result"] == "error":

    error = driver.find_element(

        By.CSS_SELECTOR, "h3[data-test='error']"

    )

    assert error.is_displayed()

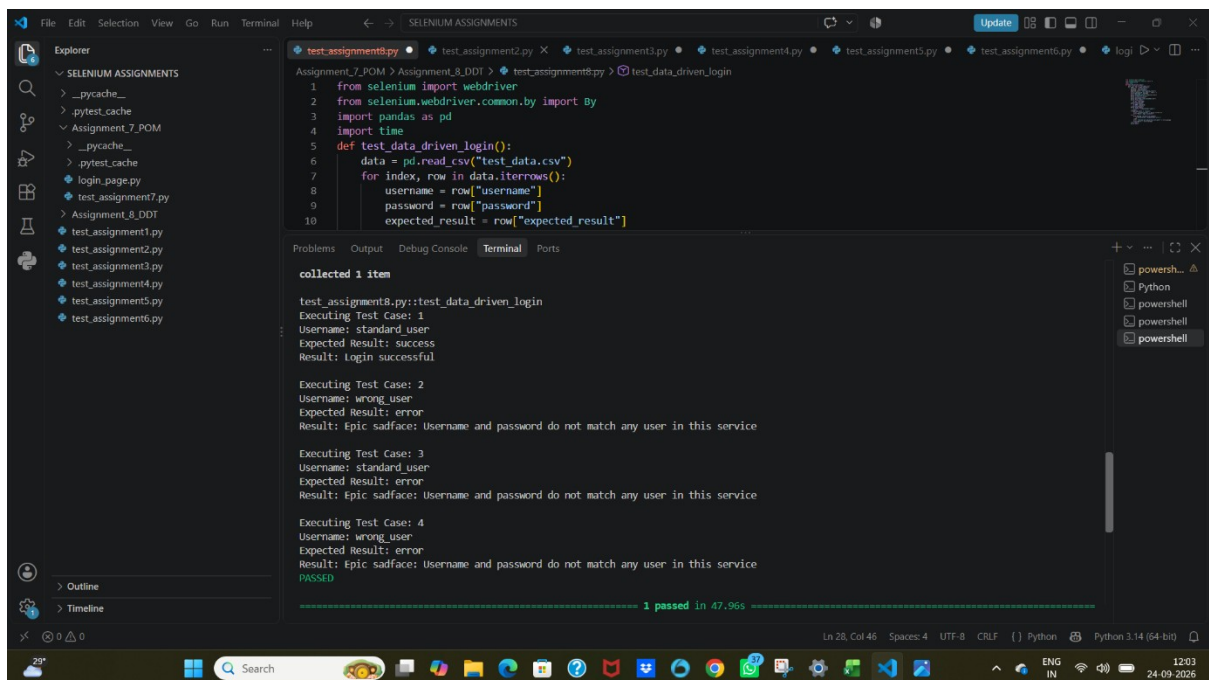
driver.quit()

```

Execution

pytest -v

6. Output with Clearly Labelled Screenshots



The screenshot shows a Visual Studio Code editor with a file explorer on the left containing a directory named 'SELENIUM ASSIGNMENTS'. The main editor window displays a Python file named 'test_assignment8.py' with the following code:

```

1 from selenium import webdriver
2 from selenium.webdriver.common.by import By
3 import pandas as pd
4 import time
5 def test_data_driven_login():
6     data = pd.read_csv("test_data.csv")
7     for index, row in data.iterrows():
8         username = row["username"]
9         password = row["password"]
10        expected_result = row["expected_result"]

```

The bottom panel shows the 'Terminal' output, which includes the following text:

```

collected 1 item

test_assignment8.py::test_data_driven_login
Executing Test Case: 1
Username: standard user
Expected Result: success
Result: Login successful

Executing Test Case: 2
Username: wrong user
Expected Result: error
Result: Epic sadface: Username and password do not match any user in this service

Executing Test Case: 3
Username: standard user
Expected Result: error
Result: Epic sadface: Username and password do not match any user in this service

Executing Test Case: 4
Username: wrong user
Expected Result: error
Result: Epic sadface: Username and password do not match any user in this service
PASSED

===== 1 passed in 47.96s =====

```

The status bar at the bottom indicates the file is at line 28, column 46, with 4 spaces, UTF-8 encoding, CRLF line endings, and is using Python 3.14 (64-bit).

PyTest Execution

Show the terminal after executing the test.

7. Result

The login automation was successfully implemented using external test data. Multiple combinations of usernames and passwords were executed, and the expected successful or error result was verified for each dataset.

8. Observation

- Test data is separated from the automation logic.
- Multiple credential combinations can be tested without duplicating the test code.
- Pandas can be used to read CSV test data.
- Assertions verify whether the actual result matches the expected result.

9. Conclusion

The experiment demonstrated **data-driven automation** using Selenium, Python, PyTest, and an external CSV file. The approach allows multiple test scenarios to be executed efficiently using a single reusable automation script.